《数据库系统原理》实验报告（5）					
题目：miniob 入门实验					
学号	2453359	姓名	李欣洋	日期	2026.4.26

实验环境：miniob

实验步骤及结果截图：

1.建议 miniob 环境（第一次搭建时已经建立完成）

```
# ./bin/observer -s miniob.sock -f ../etc/observer.ini &
#
Welcome to the OceanBase database implementation course.

Copyright (c) 2021 OceanBase and/or its affiliates.

Learn more about OceanBase at https://github.com/oceanbase/oceanbase
Learn more about MiniOB at https://github.com/oceanbase/miniob

Successfully load ../etc/observer.ini

# ./bin/obclient -s miniob.sock

Welcome to the OceanBase database implementation course.

Copyright (c) 2021 OceanBase and/or its affiliates.

Learn more about OceanBase at https://github.com/oceanbase/oceanbase
Learn more about MiniOB at https://github.com/oceanbase/miniob

miniob >
```

2.创建 Student 表

```
miniob > create table student (No int, Name char(10), Height float, Weight float);
SUCCESS
miniob > show tables;
Tables_in_SYS
student
```

3.向表中插入数据

```
miniob > insert into student values (2353123, '王海峰', 180.5, 75.5);
SUCCESS
miniob > insert into student values (2331902, '刘伟', 174.9, 63.3);
SUCCESS
miniob > insert into student values (2353074, '孙国程', 169.0, 82.5);
SUCCESS
miniob > insert into student values (2233444, '严磊', 178.3, 52.8);
SUCCESS
```

```
miniob > select * from student;
No | Name | Height | Weight
2353123 | 王海峰 | 180.5 | 75.5
2331902 | 刘伟 | 174.9 | 63.3
2353074 | 孙国程 | 169 | 82.5
2233444 | 严磊 | 178.3 | 52.8
```

4.使用 select 展示学号，身高，姓名

```
miniob > select No,Name,Height from student;
No | Name | Height
2353123 | 王海峰 | 180.5
2331902 | 刘伟 | 174.9
2353074 | 孙国程 | 169
2233444 | 严磊 | 178.3
```

5.尝试修改指定行的体重，判断能否成功，以及为什么？

```
miniob > update student set Weight=78.5 where No=2353123;
FAILURE
miniob > update student set Weight=60.0 where No=2331902;
FAILURE
```

不成功，理由如下：

在 miniob 的根目录下，src/observer/sql/executor 文件夹下，没有找到实现 update 方法的函数，即当前的 miniob 不支持 update 操作

```
# ls -l
total 96
-rw-r--r-- 1 root root 2511 Mar 23 09:49 analyze_table_executor.cpp
-rw-r--r-- 1 root root  894 Mar 23 09:49 analyze_table_executor.h
-rw-r--r-- 1 root root 3016 Mar 23 09:49 command_executor.cpp
-rw-r--r-- 1 root root  989 Mar 23 09:49 command_executor.h
-rw-r--r-- 1 root root 1404 Mar 23 09:49 create_index_executor.cpp
-rw-r--r-- 1 root root 1006 Mar 23 09:49 create_index_executor.h
-rw-r--r-- 1 root root 1454 Mar 23 09:49 create_table_executor.cpp
-rw-r--r-- 1 root root  873 Mar 23 09:49 create_table_executor.h
-rw-r--r-- 1 root root 2536 Mar 23 09:49 desc_table_executor.cpp
-rw-r--r-- 1 root root  867 Mar 23 09:49 desc_table_executor.h
-rw-r--r-- 1 root root 1936 Mar 23 09:49 execute_stage.cpp
-rw-r--r-- 1 root root 1100 Mar 23 09:49 execute_stage.h
-rw-r--r-- 1 root root 1860 Mar 23 09:49 help_executor.h
-rw-r--r-- 1 root root 5239 Mar 23 09:49 load_data_executor.cpp
-rw-r--r-- 1 root root 1017 Mar 23 09:49 load_data_executor.h
-rw-r--r-- 1 root root 4307 Mar 23 09:49 set_variable_executor.cpp
-rw-r--r-- 1 root root 1159 Mar 23 09:49 set_variable_executor.h
-rw-r--r-- 1 root root 1752 Mar 23 09:49 show_tables_executor.h
-rw-r--r-- 1 root root 2151 Mar 23 09:49 sql_result.cpp
-rw-r--r-- 1 root root 2176 Mar 23 09:49 sql_result.h
-rw-r--r-- 1 root root 1221 Mar 23 09:49 trx_begin_executor.h
-rw-r--r-- 1 root root 1482 Mar 23 09:49 trx_end_executor.h
```

6.删除记录

```
miniob > delete from student where Name = '孙国程';
SUCCESS
miniob > delete from student where Name = '严磊';
SUCCESS
miniob > select * from student;
No | Name | Height | Weight
2353123 | 王海峰 | 180.5 | 75.5
2331902 | 刘伟 | 174.9 | 63.3
```

7.Select 源码解析

第一步：语法解析

当用户输入 `select name from student where xxxx;` 时，最先由 Parser 模块接管。

- **涉及文件**：`src/observer/sql/parser/lex.l` `src/observer/sql/parser/parse.y`
- **代码逻辑**：
    1. `lex.l` 定义了 `SELECT`, `FROM`, `WHERE` 等关键字的 Token 规则。
    2. `parse.y` 中定义了 `select_stmt` 的语法推导规则。当匹配成功时，会在内存中构建出一棵抽象语法树 ，并封装为一个 `SelectSqlNode` 对象返回给上层。

第二步：计划生成

语法树只保证了“句子成分合法”，但不保证 table 真的存在。

- **涉及文件**：`src/observer/sql/stmt/select_stmt.cpp`
- **代码逻辑**： 在这一阶段，系统会根据 AST 创建一个 `SelectStmt` 对象，并调用 `resolve` 方法。它会查询内存中的元数据字典：
    1. 检查 `FROM student` 中的 `student` 表是否在数据库中被创建。
    2. 检查 `SELECT name` 和 `WHERE height` 中的字段是否属于 `student` 表。

第三步：执行引擎与算子调度

SELECT 没有单一的 select_executor.cpp（在上述第六题中的文件列表中也可以看到），而是通过算子拼装而成。

- **调度入口文件**：src/observer/sql/executor/execute_stage.cpp
- **算子目录**：src/observer/sql/operator/

第四步：模型迭代

miniob 的算子实现了经典的火山模型，每个算子都实现了三个核心 C++ 接口：open(),next(),close()。

- **代码逻辑**：
  1. 父算子调用子算子的 next() 接口。
  2. Filter Operator 调用 Table Scan 的 next() 获取一行完整数据。
  3. Filter 进行条件判断，如果满足条件，则将其作为自己的 next() 结果返回给 Project。
  4. Project Operator 裁剪掉多余的字段，如仅保留 name，最终返回给客户端。

第五步：存储层的数据读取

最底层的 Table Scan Operator 必须与真实的存储文件交互。

- **涉及文件**：src/observer/storage/common/table.cpp 及其相关文件。
- **代码逻辑**： 底层调用 Table::scan_record() 或通过索引管理器读取数据页。

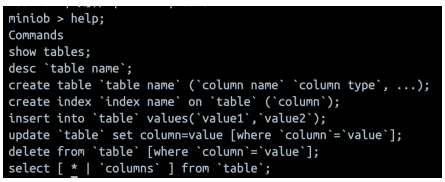
出现的问题：

1.在建表之前准备建立一个数据库来存储这张表，发现报错

2.help 查看命令发现命令中包含 update，所以认为 update 能够成功，但是实际上 update 无法成功

解决方案：

1.输入 help 命令查看之后，发现 minion 目前还没有实现 creat databse 的方法



2.查阅大模型得知，help 中显示 update 命令不代表 miniob 实现了 update 方法，所以去系统根目录下查看文件夹发现的确没有实现 update 的方法，因此导致 update 发生报错，导致失败